【RabbitMQ】

主要内容

1. AMQP 简介
2. RabbitMQ 简介
3. RabbitMQ 原理
4. Erlang 安装
5. 安装 RabbitMQ
6. RabbitMQ 账户管理
7. 交换器

学习目标

知识点	要求
AMQP 简介	掌握
RabbmitMQ 简介	掌握
RabbitMQ 原理	掌握
Erlang 安装	掌握
安装 RabbitMQ	掌握
RabbitMQ 账户管理	掌握
交换器	掌握

一、 AMQP 简介

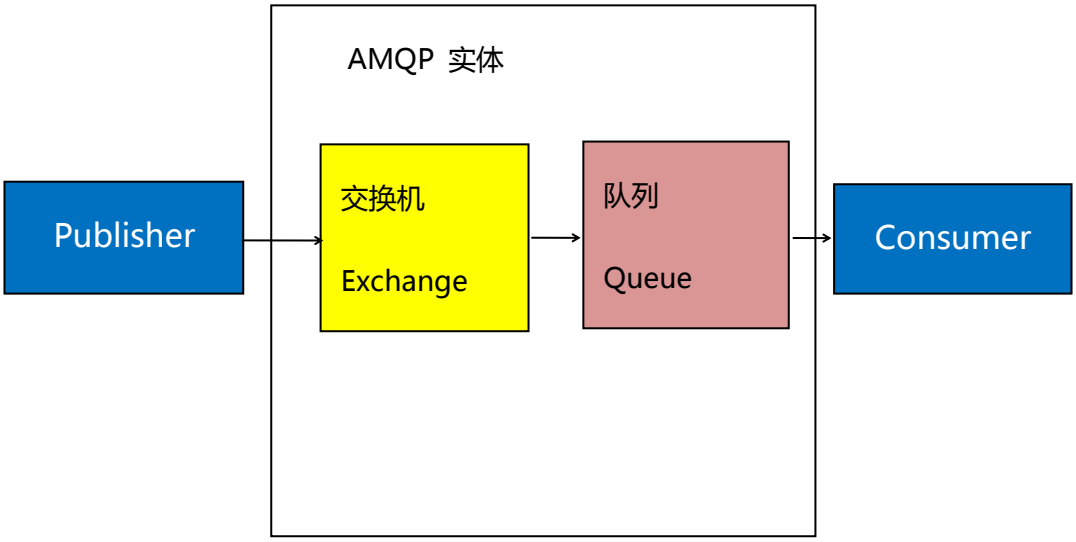
1 AMQP 是什么?

AMQP(Advanced Message Queuing Protocol,高级消息队列协议)是进程之间传递异步消息的网络协议。

2 AMQP 工作过程

发布者(Publisher)发布消息(Message),经过交换机(Exchange)，交换机根据路由规则

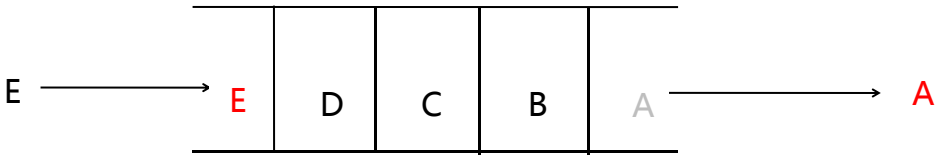
将收到消息分发给交换机绑定的队列(Queue)，最后 AMQP 代理会将消息投递给订阅了此队列的消费者，或者消费者按照需求自行获取。



3 队列

队列是数据结构中概念。数据存储在一个队列中，数据是有顺序的，先进的先出，后进后出。其中一侧负责进数据，另一次负责出数据。

MQ（消息队列）很多功能都是基于此队列结构实现的



二、 RabbitMQ 简介

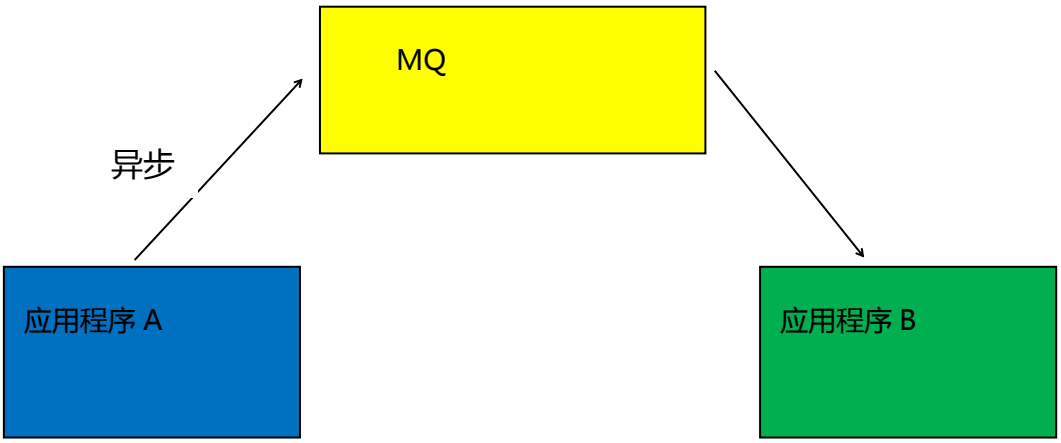
1 RabbitMQ 介绍

RabbitMQ 是由 Erlang 语言编写的基于 AMQP 的消息中间件。而消息中间件作为分布式系统重要组件之一，可以解决应用耦合，异步消息，流量削峰等问题。

1.1 解决应用耦合

1.1.1 不使用 MQ 时

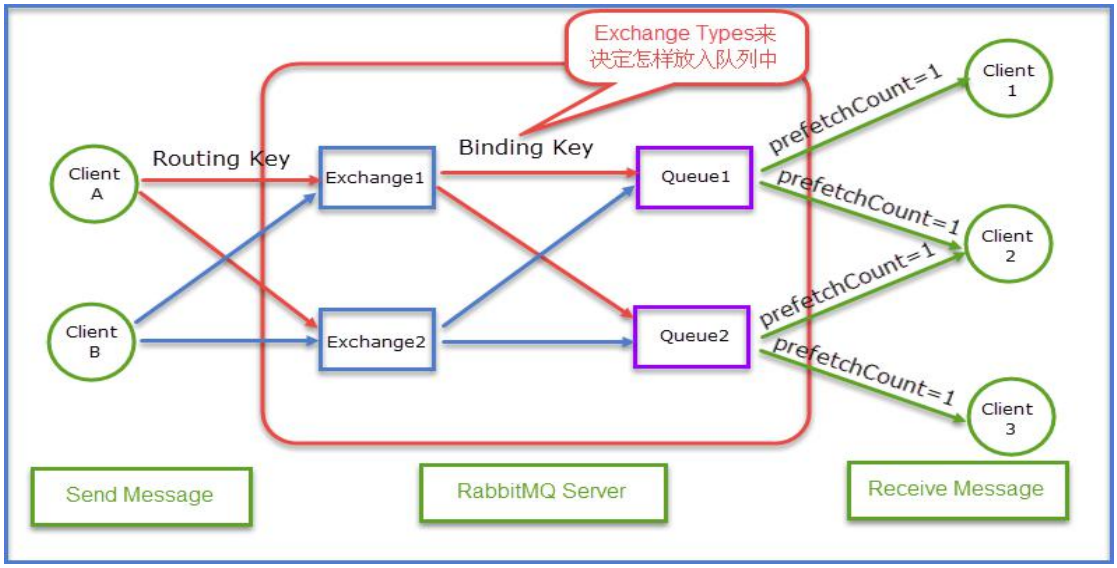
1.1.2 使用 MQ 解决耦合



2 RabbitMQ 适用场景

排队算法、秒杀活动、消息分发、异步处理、数据同步、处理耗时任务、流量销峰等。

三、 RabbitMQ 原理



1.Message
消息。消息是不具名的，它由消息头消息体组成。消息体是不透明的，而消息头则由一系列可选属性组成，这些属性包括：routing-key(路由键)、priority(相对于其他消息的优先权)、delivery-mode(指出消息可能持久性存储)等。
2.Publisher
消息的生产者。也是一个向交换器发布消息的客户端应用程序。
3.Consumer
消息的消费者。表示一个从消息队列中取得消息的客户端应用程序。
4.Exchange
交换器。用来接收生产者发送的消息并将这些消息路由给服务器中的队列。 三种常用的交换器类型 1. direct(发布与订阅 完全匹配) 2. fanout(广播)

3. topic(主题，规则匹配)

5.Binding

绑定。用于消息队列和交换器之间的关联。一个绑定就是基于路由键将交换器和消息队列连接起来的路由规则，所以可以将交换器理解成一个由绑定构成的路由表。

6.Queue

消息队列。用来保存消息直到发送给消费者。它是消息的容器，也是消息的终点。一个消息可投入一个或多个队列。消息一直在队列里面，等待消费者链接到这个队列将其取走。

7.Routing-key

路由键。RabbitMQ 决定消息该投递到哪个队列的规则。（也可以理解为队列的名称，路由键是 key，队列是 value）

队列通过路由键绑定到交换器。

消息发送到 MQ 服务器时，消息将拥有一个路由键，即便是空的，RabbitMQ 也会将其和绑定使用的路由键进行匹配。

如果相匹配，消息将会投递到该队列。

如果不匹配，消息将会进入黑洞。

8.Connection

链接。指 rabbit 服务器和服务建立的 TCP 链接。

9.Channel

信道。

1，Channel 中文叫做信道，是 TCP 里面的虚拟链接。例如：电缆相当于 TCP，信道

是一个独立光纤束，一条 TCP 连接上创建多条信道是没有问题的。

2，TCP 一旦打开，就会创建 AMQP 信道。

3，无论是发布消息、接收消息、订阅队列，这些动作都是通过信道完成的。

10.Virtual Host

虚拟主机。表示一批交换器，消息队列和相关对象。虚拟主机是共享相同的身份认证和加密环境的独立服务器域。每个 vhost 本质上就是一个 mini 版的 RabbitMQ 服务器，拥有自己的队列、交换器、绑定和权限机制。vhost 是 AMQP 概念的基础，必须在链接时指定，RabbitMQ 默认的 vhost 是/

11.Borker

表示消息队列服务器实体。

交换器和队列的关系

交换器是通过路由键和队列绑定在一起的，如果消息拥有的路由键跟队列和交换器的路由键匹配，那么消息就会被路由到该绑定的队列中。

也就是说，消息到队列的过程中，消息首先会经过交换器，接下来交换器在通过路由键匹配分发消息到具体的队列中。

路由键可以理解为匹配的规则。

RabbitMQ 为什么需要信道？为什么不是 TCP 直接通信？

1. TCP 的创建和销毁开销特别大。创建需要 3 次握手，销毁需要 4 次分手。
2. 如果不用信道，那应用程序就会以 TCP 链接 Rabbit，高峰时每秒成千上万条链接会造成资源巨大的浪费，而且操作系统每秒处理 TCP 链接数也是有限制的，必定造成性能瓶颈。

3. 信道的原理是一条线程一条通道，多条线程多条通道同用一条 TCP 链接。一条 TCP 链接可以容纳无限的信道，即使每秒成千上万的请求也不会成为性能的瓶颈。

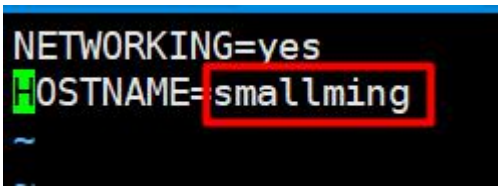
四、 Erlang 安装

RabbitMQ 是使用 Erlang 语言编写的，所以需要先配置 Erlang

1 修改主机名

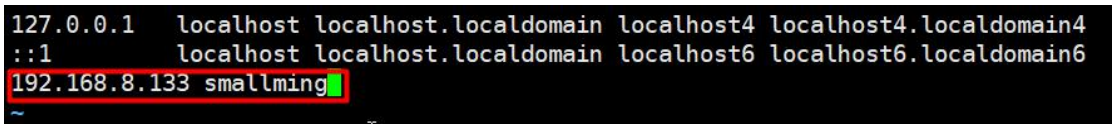
RabbitMQ 是通过主机名进行访问的，必须指定能访问的主机名。

vim /etc/sysconfig/network



vim /etc/hosts

新添加了一行，前面为服务器 ip，空格后面添加计算机主机名



2 安装依赖

yum -y install make gcc gcc-c++ kernel-devel m4 ncurses-devel openssl-devel unixODBC unixODBC-devel

3 上传并解压

上传 otp_src_22.0.tar.gz 到/usr/local/tmp 目录中，进入目录并解压。

解压时注意，此压缩包不具有 gzip 属性，解压参数没有 z，只有 xf

cd /usr/local/tmp

tar xf otp_src_22.0.tar.gz

4 配置参数

先新建/usr/local/erlang 文件夹，作为安装文件夹

#mkdir -p /usr/local/erlang

进入文件夹

cd otp_src_22.0

配置参数

./configure --prefix=/usr/local/erlang --with-ssl --enable-threads --enable-smp-support --enable-kernel-poll --enable-hipe --without-javac

5 编译并安装

编译

make

安装

make install

6 修改环境变量

修改/etc/profile 文件

#vim /etc/profile

在文件中添加下面代码

```
export PATH=$PATH:/usr/local/erlang/bin
```

运行文件，让修改内容生效

source /etc/profile

7 查看配置是否成功

erl -version

```
[root@localhost otp_src_22.0]# erl -version
Erlang (SMP,ASYNC_THREADS,HIPE) (BEAM) emulator version 10.4
```

五、 安装 RabbitMQ

1 上传并解压

上传 rabbitmq-server-generic-unix-3.7.17.tar.xz 到/usr/loca/tmp 中

cd /usr/local/tmp

tar xf rabbitmq-server-generic-unix-3.7.17.tar.xz

2 复制到 local 下

复制解压文件到/usr/local 下，命名为 rabbitmq

cp -r rabbitmq_server-3.7.17 /usr/local/rabbitmq

3 配置环境变量

vim /etc/profile

在文件中添加

```
export PATH=$PATH:/usr/local/rabbitmq/sbin
```

解析文件

source /etc/profile

4 开启 web 管理插件

进入 rabbitmq/sbin 目录

cd /usr/local/rabbitmq/sbin

查看插件列表

./rabbitmq-plugins list

生效管理插件

./rabbitmq-plugins enable rabbitmq_management

5 后台运行

启动 rabbitmq。

#./rabbitmq-server -detached

停止命令，如果无法停止，使用 kill -9 进程号进行关闭

./rabbitmqctl stop_app

启动成功信息如下：

```
[root@192 rabbitmq]# sbin/rabbitmq-server

  ##  ##
  ##  ##      RabbitMQ 3.7.17. Copyright (C) 2007-2019 Pivotal Software, Inc.
  ##########  Licensed under the MPL.  See https://www.rabbitmq.com/
  ######  ##
  ##########  Logs: /usr/local/rabbitmq/var/log/rabbitmq/rabbit@bjsxt-mq.log
                    /usr/local/rabbitmq/var/log/rabbitmq/rabbit@bjsxt-mq_upgrade.log

              Starting broker...
 completed with 3 plugins.
```

5.1 启动错误解决

如果启动 RabbitMQ 发生下述错误，可以提供环境配置文件，解决。环境配置文件命名为: rabbitmq-env.conf。所在位置是： $rabbitmq_home/etc/rabbitmq/目录。内如是： HOSTNAME=主机名称

```
[root@192 rabbitmq]# sbin/rabbitmq-server
ERROR: epmd error for host 192: badarg (unknown POSIX error)
```

6 查看 web 管理界面

默认可以在安装 rabbitmq 的电脑上通过用户名：guest 密码 guest 进行访问 web 管理界面

端口号：15672（放行端口，或关闭防火墙）

在虚拟机浏览器中输入：

http://localhost:15672

六、 RabbitMq 账户管理

1 创建账户

语法：./rabbitmqctl add_user username password

cd /usr/local/rabbitmq/sbin

./rabbitmqctl add_user admin admin123

2 给用户授予管理员角色

其中 smallming 为新建用户的用户名

./rabbitmqctl set_user_tags admin administrator

3 给用户授权

“/” 表示 RabbitMQ 根虚拟主机

admin 表示用户名

".*" ".*" ".*" 表示完整权限

./rabbitmqctl set_permissions -p "/" admin ".*" ".*" ".*"

4 登录

使用新建账户和密码在 windows 中访问 rabbitmq 并登录

在浏览器地址栏输入：

http://ip:15672/

用户名：admin

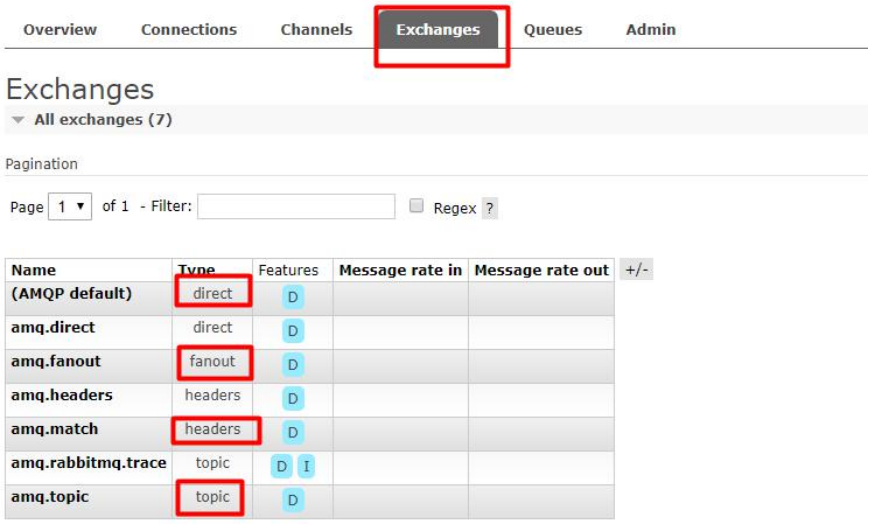
密码：admin123

七、 Exchange 交换器（交换机）

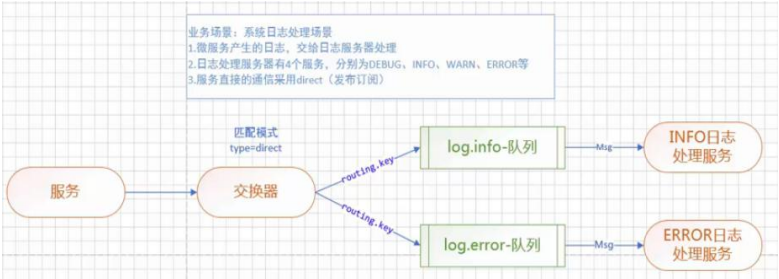
交换器负责接收客户端传递过来的消息，并转发到对应的队列中。在 RabbitMQ 中支持四种交换器

1. Direct Exchange：直连交换器（默认）
2. Fanout Exchange：扇形交换器
3. **Topic Exchange**：主题交换器
4. Header Exchange：首部交换器。

在 RabbitMq 的 Web 管理界面中 Exchanges 选项卡就可以看见这四个交换器。



1 direct 交换器



direct 交换器是 RabbitMQ 默认交换器。默认会进行公平调度。所有接受者依次从消息队列中获取值。Publisher 给哪个队列发消息，就一定是给哪个队列发送消息。对交换器绑定的其他队列没有任何影响。

（代码演示）一个队列需要绑定多个消费者

需要使用注解/API：

org.springframework.amqp.core.Queue:队列

AmqpTemplate：操作 RabbitMQ 的接口。负责发送或接收消息

@RabbitListener(queues = "") 注解某个方法为接收消息方法

1.1 代码实现

1.1.1 新建项目 Publisher

1.1.1.1 添加依赖

```
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>2.2.2.RELEASE</version>
</parent>


<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-amqp</artifactId>
    </dependency>
</dependencies>
```

1.1.1.2 编写配置文件

新建 application.yml.

host:默认值 localhost

username 默认值：guest

password 默认值：guest

```
spring:
  rabbitmq:
    host: 192.168.52.132
    username: admin
    password: admin123
```

1.1.1.3 编写消息类

正式开发中，消息类型应该定义在一个独立工程中，使用工程依赖的方式导入到 Producer 和 Consumer 工程中，课上环境为同样的代码在 Producer 和 Consumer 工程中重复定义。

```
package com.bjsxt.message;


import java.io.Serializable;
import java.util.Date;


public class LogMessage implements Serializable {
```

```
    private Long id;
    private String msg;
    private String logLevel;
    private String serviceType;
    private Date createTime;
    private Long userId;
    public LogMessage() {
        super();
    }
    public LogMessage(Long id, String msg, String logLevel, String serviceType, Date createTime, Long userId) {
        super();
        this.id = id;
        this.msg = msg;
        this.logLevel = logLevel;
        this.serviceType = serviceType;
        this.createTime = createTime;
        this.userId = userId;
    }
    @Override
    public String toString() {
```

```
        return "LogMessage [id=" + id + ", msg=" + msg + ", logLevel=" +
logLevel + ", serviceType=" + serviceType
                + ", createTime=" + createTime + ", userId=" + userId + "]";
    }
    // 省略 getter/setter 方法
}
```

1.1.1.4 编写生产者

```
package com.bjsxt;


import org.springframework.amqp.core.AmqpTemplate;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;


import com.bjsxt.message.LogMessage;


/**
 * 消息发送者
 */
@Component
public class Sender {
@Autowired
```

```
        private AmqpTemplate rabbitAmqpTemplate;

        /*
         * 发送消息的方法
         */
        public void sendError(LogMessage msg){
            //向消息队列发送消息
            //参数一：交换器名称。
            //参数二：路由键
            //参数三：消息
            this.rabbitAmqpTemplate.convertAndSend("exchange_direct", "key1",
msg);
        }

        public void sendInfo(LogMessage msg){
            this.rabbitAmqpTemplate.convertAndSend("exchange_direct", "key2",
msg);
        }
    }
```

1.1.1.5 编写测试类

SpringBoot 整合 Spring-AMQP 后包含内置对象 AmqpTemplate

```
package com.bjsxt.test;
```

```
import java.util.Date;

import java.util.Random;


import org.junit.Test;

import org.junit.runner.RunWith;

import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.boot.test.context.SpringBootTest;

import org.springframework.test.context.junit4.SpringRunner;


import com.bjsxt.Sender;

import com.bjsxt.SpringbootServerApplication;

import com.bjsxt.message.LogMessage;


/**

 * 消息队列测试类

 */

@RunWith(SpringRunner.class)

@SpringBootTest(classes=SpringbootServerApplication.class)

public class QueueTest {
```

```
        @Autowired
        private Sender sender;


        /*
         * 测试消息队列
         */
        @Test
        public void testSend()throws Exception{
            Random r = new Random();
            for(long i = 0; i < 100; i++){
                Thread.sleep(1000);
                if(r.nextInt(1000)%2 == 0) {
                    this.sender.sendError(new LogMessage(i, "test error log",
"error", "订单服务", new Date(), i*2));
                }else{
                    this.sender.sendInfo(new LogMessage(i, "test info log", "info",
"订单服务", new Date(), i*2));
                }
            }
        }
    }
```

1.1.2 创建 Consumer

新建项目 consumer

1.1.2.1 添加依赖

和 Publisher 一样

1.1.2.2 编写配置文件

和 Publisher 一样

1.1.2.3 编写监听器方法

注意：

类上要有@Componet 或使用 Configuration 初始化，项目启动时此类生效。

@RabbitListener 监听指定队列。

```
package com.bjsxt;


import org.springframework.amqp.core.ExchangeTypes;
import org.springframework.amqp.rabbit.annotation.Exchange;
import org.springframework.amqp.rabbit.annotation.Queue;
import org.springframework.amqp.rabbit.annotation.QueueBinding;
import org.springframework.amqp.rabbit.annotation.RabbitHandler;
import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.stereotype.Component;
```

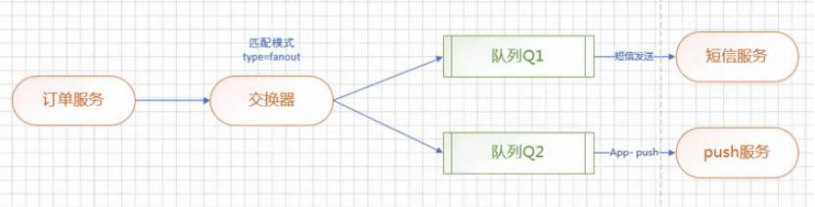
```
import com.bjsxt.message.LogMessage;


/**
 * 消息接收者
 * @author Administrator
 * @RabbitListener bindings:绑定队列
 * @QueueBinding    value:绑定队列的名称
 *                  exchange:配置交换器
 *                  key:路由键
 *
 * @Queue value:配置队列名称
 *        autoDelete:是否是一个可删除的临时队列
 *
 * @Exchange value:为交换器起个名称
 *           type:指定具体的交换器类型
 */
@Component
@RabbitListener(
        bindings=@QueueBinding(
                value=@Queue(value="queue-log1",autoDelete="false"),
```

```
exchange=@Exchange(value="exchange-name",type=ExchangeTypes.DIRECT)
,
                    key="key1"
            )
        )
public class ErrorReceiver {

    /**
     * 接收消息的方法。采用消息队列监听机制
     * @param msg
     */
    @RabbitHandler
    public void process(LogMessage msg){
        System.out.println("Error.........receiver: "+msg);
    }
}
```

1.1.2.4 新建启动类

略。启动后此项目一直处于运行状态。属于长连接。

2 fanout 交换器



扇形交换器，实际上做的事情就是广播，fanout 会把消息发送给所有的绑定在当前交换器上的队列。对应 Consumer 依然采用公平调度方式。

（代码演示）一个交换器需要绑定多个队列

需要使用注解/API：

FanoutExchange：fanout 交换器

Binding：绑定交换器和队列

BindingBuilder：Binding 的构建器

amq.fanout:内置 fanout 交换器名称

2.1 代码

2.1.1 Publisher

```
package com.bjsxt.fanout;


import org.springframework.amqp.core.AmqpTemplate;

import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.beans.factory.annotation.Value;

import org.springframework.stereotype.Component;
```

```
/**
 * 消息发送者
 */
@Component
public class FanoutSender {

    @Autowired
    private AmqpTemplate rabbitAmqpTemplate;

    /*
     * 发送消息的方法
     */
    public void send(String msg){
        this.rabbitAmqpTemplate.convertAndSend("exchange-name","", msg);
    }
}
```

2.1.2 编写发送方法

第二个参数 routingKey 对于 fanout 没有意义的

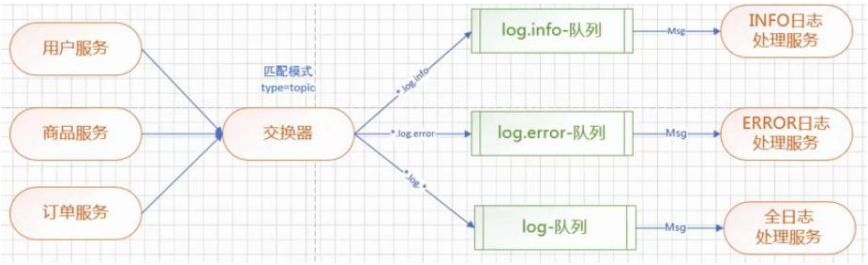
```
/**
 * 消息队列测试类
 */
```

```
@RunWith(SpringRunner.class)

@SpringBootTest(classes=SpringbootServerApplication.class)

public class QueueTest {



    @Autowired

    private FanoutSender fanoutSender;



    /*

     * 测试 Fanout 消息队列

     */

    @Test

    public void testFanout(){

        this.fanoutSender.send("Fanout - Hello RabbitMQ");

    }

}
```

2.1.3 Consumer

Consumer 代码和 Direct 完全相同。一个队列给一个监听方法即可。

3 topic 交换器



允许在路由键（RoutingKey）中出现匹配规则。

路由键的写法和包写法相同。com.bjsxt.xxxx.xxx 格式。

在绑定时可以带有下面特殊符号，中间可以出现:

*：代表一个单词（两个.之间内容）

#：0 个或多个字符

接收方依然是公平调度，同一个队列中内容轮换获取值。

需要使用注解/API：

TopicExchange：Topic 交换器

amq.topic:内置 topic 交换器名称

3.1 代码

3.1.1 Publisher

3.1.1.1 发送消息

```
package com.bjsxt.topic;


import org.springframework.amqp.core.AmqpTemplate;
```

```
import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.beans.factory.annotation.Value;

import org.springframework.stereotype.Component;


/**

 * 消息发送者

 */

@Component

public class UserSender {


    @Autowired

    private AmqpTemplate rabbitAmqpTemplate;


    /*

     * 发送消息的方法

     */

    public void sendInfo(String msg){

        this.rabbitAmqpTemplate.convertAndSend("exchange-name","k.info",
"user.log.info....."+msg);

    }

    public void sendDebug(String msg){
```

```
    this.rabbitAmqpTemplate.convertAndSend("exchange-name","k.debug",
"user.log.debug....."+msg);
        }
        public void sendWarn(String msg){

    this.rabbitAmqpTemplate.convertAndSend("exchange-name","k.warn",
"user.log.warn....."+msg);
        }
        public void sendError(String msg){

    this.rabbitAmqpTemplate.convertAndSend("exchange-name","k.error",
"user.log.error....."+msg);
        }
    }
```

3.1.2 Consumer

```
    package com.bjsxt.topic;

    import org.springframework.amqp.core.ExchangeTypes;
```

```
import org.springframework.amqp.rabbit.annotation.Exchange;

import org.springframework.amqp.rabbit.annotation.Queue;

import org.springframework.amqp.rabbit.annotation.QueueBinding;

import org.springframework.amqp.rabbit.annotation.RabbitHandler;

import org.springframework.amqp.rabbit.annotation.RabbitListener;

import org.springframework.stereotype.Component;


@Component

@RabbitListener(

                bindings=@QueueBinding(

                        value=@Queue(value="q-info",autoDelete="true"),


exchange=@Exchange(value="exchange-name",type=ExchangeTypes.TOPIC),

                        key="*.info"

                )

            )

public class TopicInfoReceiver {

    @RabbitHandler

    public void process(String msg){

        System.out.println(".....Info........receiver: "+msg);

    }
```

```
}
```

```
package com.bjsxt.topic;


import org.springframework.amqp.core.ExchangeTypes;
import org.springframework.amqp.rabbit.annotation.Exchange;
import org.springframework.amqp.rabbit.annotation.Queue;
import org.springframework.amqp.rabbit.annotation.QueueBinding;
import org.springframework.amqp.rabbit.annotation.RabbitHandler;
import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.stereotype.Component;


@Component
@RabbitListener(
        bindings=@QueueBinding(
                value=@Queue(value="q-error",autoDelete="true"),


exchange=@Exchange(value="exchange-name",type=ExchangeTypes.TOPIC),
                key="*.error"
        )
    )
```

```
public class TopicErrorReceiver {


    @RabbitHandler

    public void process(String msg){

        System.out.println(".....Error.....receiver: "+msg);

    }

}
```

```
package com.bjsxt.topic;


import org.springframework.amqp.core.ExchangeTypes;

import org.springframework.amqp.rabbit.annotation.Exchange;

import org.springframework.amqp.rabbit.annotation.Queue;

import org.springframework.amqp.rabbit.annotation.QueueBinding;

import org.springframework.amqp.rabbit.annotation.RabbitHandler;

import org.springframework.amqp.rabbit.annotation.RabbitListener;

import org.springframework.stereotype.Component;


@Component

@RabbitListener(

            bindings=@QueueBinding(
```

```
                    value=@Queue(value="q-all",autoDelete="true"),

exchange=@Exchange(value="exchange-name",type=ExchangeTypes.TOPIC),
                    key="*"
            )
        )
public class TopicLogsReceiver {
    @RabbitHandler
    public void process(String msg){
        System.out.println(".....All.....receiver: "+msg);
    }
}
```

八、 传递对象类型参数

如果消息是对象类型，此对象的类型必须进行序列化，且需要给定序列化值

```
public class People implements Serializable  {
    public static final long serialVersionUID=1L;
}
```